

期末试题答案汇总

目录

1	编译原理期末试题答案汇总
---	--------------

2

1 编译原理期末试题答案汇总

1.1 使用方法

这份文件按“同类题”整理。看到题目先匹配标签，再套答案模板。

标签：

[自动机] 正则 -> NFA -> DFA -> 最小化
 [LL] 消除左递归 -> FIRST -> FOLLOW -> 预测表
 [LR] 项目 -> closure -> goto -> 分析表
 [ANTLR] 优先级、结合性、语法树
 [SDD] 属性与语义规则
 [TAC] 三地址码、控制流回填
 [SSA] 支配、支配边界、phi
 [调度] 依赖图、寄存器重命名、拓扑序
 [符号执行] 路径条件、CNF、Tseitin
 [Andersen] points-to 约束与闭包

1.2 通用答案模板

1.2.1 [自动机] Thompson

字符 a: start --a--> end
 epsilon: start --epsilon--> end
 A|B: 新 start epsilon 到 A/B 起点, A/B 终点 epsilon 到新 end
 AB: A 终点 epsilon 到 B 起点
 A*: 新 start epsilon 到 A 起点和新 end, A 终点 epsilon 到 A 起点和新 end

1.2.2 [自动机] 子集构造

Start = epsilon-closure({NFA start})
 对每个 DFA 状态 S 和字符 a:
 T = epsilon-closure(move(S, a))
 记录 S --a--> T
 含 NFA 终态的集合是 DFA 终态。

1.2.3 [自动机] DFA 最小化

P0 = {终态集合, 非终态集合}
 按每个输入字符的后继所在组拆分。
 重复直到分组不再变化。
 每个分组变成一个最小 DFA 状态。

1.2.4 [LL]

1. 消除左递归。
2. 计算 FIRST。
3. 计算 FOLLOW。
4. 对 $A \rightarrow \alpha$:
 $\text{FIRST}(\alpha) - \{\epsilon\}$ 填 $M[A, a]$
 若 $\epsilon \in \text{FIRST}(\alpha)$, $\text{FOLLOW}(A)$ 填 $M[A, b]$
5. 表中无多重入口就是 LL(1)。

1.2.5 [LR]

closure:

若 $[A \rightarrow \alpha \cdot B \beta, a]$ 在 I 中,

对 $B \rightarrow \gamma$ 和 $b \in \text{FIRST}(\beta a)$, 加入 $[B \rightarrow \cdot \gamma, b]$ 。

goto(I,X):

closure($\{[A \rightarrow \alpha X \cdot \beta, a]\}$)

归约:

$[A \rightarrow \alpha \cdot, a]$ 只在 $\text{action}[a]$ 中填 reduce $A \rightarrow \alpha$ 。

1.2.6 [TAC] 回填

B.truelist, B.falselist, S.nextlist
 makelist(i), merge(p,q), backpatch(p,L)

while:

M1 = 条件入口

翻译 B

M2 = 循环体入口

backpatch(B.truelist, M2)

翻译 S1

emit(goto M1)

S.nextlist = B.falselist

1.2.7 [Andersen]

$y = \&x : x \text{ in } \text{pts}(y)$

$y = x : \text{pts}(x) \text{ subset } \text{pts}(y)$

$*y = x : \text{for } v \text{ in } \text{pts}(y), \text{pts}(x) \text{ subset } \text{pts}(v)$

$y = *x : \text{for } v \text{ in } \text{pts}(x), \text{pts}(v) \text{ subset } \text{pts}(y)$

1.3 2025

1.3.1 题 1 [自动机] 正则表达式到 NFA、DFA、最小化

题干要求：

把题面正则表达式转换成 NFA。

把 NFA 转换成 DFA。

最小化 DFA。

答案写法：

1. 按括号层次拆正则表达式。
2. 对 `epsilon`、字符、选择、连接、星闭包分别套 Thompson 模板。
3. 给 NFA 状态编号。
4. 写每个 DFA 状态对应的 NFA 状态集合。
5. 对每个输入字符写 `move` 和 `epsilon-closure`。
6. 含 NFA 终态的 DFA 状态标为终态。
7. 最小化时先分终态/非终态，再按转移目标组拆分。

评分点：

NFA 结构正确。

DFA 状态集合对应关系写清。

DFA 终态判断正确。

最小化分组过程写清。

1.3.2 题 2 [LL] 消除左递归、FIRST/FOLLOW、预测表

题干：

```
A -> B C
B -> B P | r
C -> t | u
```

消除左递归：

```
A -> B C
B -> r B'
B' -> P B' | epsilon
C -> t | u
```

FIRST:

```

FIRST(A)  = {r}
FIRST(B)  = {r}
FIRST(B') = {P, epsilon}
FIRST(C)  = {t, u}

```

FOLLOW:

```

FOLLOW(A) = {$}
FOLLOW(B) = {t, u}
FOLLOW(B') = {t, u}
FOLLOW(C) = {$}

```

预测表:

```

M[A,r]  = A -> B C
M[B,r]  = B -> r B'
M[B',P] = B' -> P B'
M[B',t] = B' -> epsilon
M[B',u] = B' -> epsilon
M[C,t]  = C -> t
M[C,u]  = C -> u

```

结论:

表中无多重入口, 消除左递归后的文法是 LL(1)。

1.3.3 题 3 [调度] 指令调度

目的:

在不改变程序语义的前提下重排指令, 减少流水线停顿, 提高并行执行效率。

依赖定义:

True dependence / RAW:

i 写 x, j 读 x, i 必须在 j 前。

Anti dependence / WAR:

i 读 x, j 写 x, j 不能提前破坏 i 读到的旧值。

Output dependence / WAW:

i 写 x, j 写 x, 二者顺序影响最终写入值。

依赖图:

每条指令一个节点。

RAW 用实线。

WAR 和 WAW 用虚线。

边从先执行者指向后执行者。

寄存器重命名：

WAR 和 WAW 能通过给后写入者换新寄存器消除。

RAW 不能通过重命名消除。

调度：

删去已通过重命名消除的 WAR/WAW。

对剩余依赖图给出拓扑序。

拓扑序就是合法调度序列。

1.3.4 题 4 [SSA] 支配问题

定义：

$\text{Dom}(n)$ ：

从入口到 n 的每条路径都经过的节点集合。

$\text{PostDom}(n)$ ：

从 n 到出口的每条路径都经过的节点集合。

$\text{Dom Frontier}(n)$ ：

n 支配某个前驱，但不严格支配该节点的节点集合。

支配关系图是森林：

除入口外，每个节点有唯一直接支配者 idom 。

idom 严格支配该节点。

严格支配关系无环。

因此每个节点至多一个父节点且无环，构成树；多入口时构成森林。

构建支配关系图：

1. 对每个节点求 Dom 集。
2. 在 $\text{Dom}(n) - \{n\}$ 中找最靠近 n 的节点作为 $\text{idom}(n)$ 。
3. 画 $\text{idom}(n) \rightarrow n$ 。

1.3.5 题 5 [符号执行] 敏感性、CNF、Tseitin

敏感性:

Path-sensitive: 区分不同控制流路径。
Flow-sensitive: 区分程序点和语句顺序。
Context-sensitive: 区分函数调用上下文。

CNF:

CNF 是子句的合取, 每个子句是文字的析取。
 (a or not b) and (c or d or not e)

Tseitin:

1. 给每个子表达式引入新变量。
2. 写新变量与子表达式等价的 CNF 约束。
3. 加入根变量为真的子句。
4. 得到与原公式同时可满足的 CNF。

同时可满足证明:

$F1 = (a_0 \text{ or } \dots \text{ or } a_n \text{ or } c) \text{ and } (b_0 \text{ or } \dots \text{ or } b_m \text{ or not } c)$
 $F2 = (a_0 \text{ or } \dots \text{ or } a_n \text{ or } b_0 \text{ or } \dots \text{ or } b_m)$

$F1 \rightarrow F2$:

若 $c=false$, 则第一子句要求某个 $a_i=true$ 。
 若 $c=true$, 则第二子句要求某个 $b_j=true$ 。
 因此 $F2=true$ 。

$F2 \rightarrow F1$:

若某个 $a_i=true$, 令 $c=true$ 。
 若某个 $b_j=true$, 令 $c=false$ 。
 因此 $F1=true$ 。

所以 $F1$ 与 $F2$ 同时可满足。

1.3.6 题 6 [Andersen] 指针别名

题干结构:

```
p = &a;
q = &b;
*p = q;
r = &c;
s = p;
t = *p;
*s = r;
```

约束:

```
a in pts(p)
b in pts(q)
for v in pts(p), pts(q) subset pts(v)
c in pts(r)
pts(p) subset pts(s)
for v in pts(p), pts(v) subset pts(t)
for v in pts(s), pts(r) subset pts(v)
```

闭包:

```
pts(p) = {a}
pts(q) = {b}
由 *p=q 得 pts(a) 包含 {b}
pts(r) = {c}
由 s=p 得 pts(s) 包含 {a}
由 t=*p 得 pts(t) 包含 pts(a), 先得到 {b}
由 *s=r 得 pts(a) 包含 {c}
pts(a) = {b,c}
pts(t) = {b,c}
```

1.4 2023

1.4.1 题 1 [自动机] $a((\mid \square))^*$

答案写法:

1. ``a`` 画一条字符边。
2. ``(`` 与 ``[]`` 分别看作两个后缀 `token` 序列。
3. ``(( | [])`` 用选择模板。
4. 外层 ``*`` 用闭包模板。
5. 最后与前面的 ``a`` 连接。
6. 子集构造时写清每个 DFA 状态对应的 NFA 状态集合。

1.4.2 题 2 [LL] C 声明文法

题干:

```
D -> D()
D -> D[]
D -> *D
D -> (D)
D -> id
```

FIRST:

$\text{FIRST}(D) = \{*, (, \text{id}\}$

FOLLOW:

$\text{FOLLOW}(D) = \{ (, [,), \$ \}$

结论:

原文法有直接左递归 $D \rightarrow D()$ 和 $D \rightarrow D[]$ 。

预测分析表会在同一入口填入多条产生式。

因此原文法不是 $\text{LL}(1)$ 。

1.4.3 题 3 [LR] C 声明文法

答题模板:

1. 写增广文法 $D' \rightarrow D$ 。
2. 从 $[D' \rightarrow \cdot D, \$]$ 开始做 closure。
3. 对每个终结符和非终结符做 goto。
4. shift 项按 goto 填 action。
5. reduce 项按 FOLLOW 或 $\text{LR}(1)$ 向前看符号填 action。
6. 若同一 action 表格有 shift/reduce 或 reduce/reduce, 则说明冲突。

SLR 冲突解释:

SLR 使用 FOLLOW(D) 放置归约。

D 后面既能接后缀 $()$ 、 $[]$, 也能结束或遇到右括号。

FOLLOW 过粗, 会把归约放到产生冲突的位置。

消解方式:

后缀 $()$ 与 $[]$ 优先级高于前缀 $*$ 。

遇到 $()$ 或 $[]$ 时移入, 先形成后缀结构。

遇到 $\$$ 或 $)$ 时再归约。

1.4.4 题 4 [ANTLR] C 声明优先级上升

核心优先级:

postfix $()$ 和 $[]$ 优先级高。

prefix $*$ 优先级低于 postfix。

括号 (D) 改变结合结构。

id 是基本项。

输入:

```
(*(a)())[]
```

解释写法:

1. 最内层 **a** 是基本项。
2. **(**a**)** 通过括号把 **pointer** 结构绑定成一个整体。
3. **(**a**)()** 先应用 **postfix function-call**。
4. ***(**a**)()** 形成 **pointer to function-returning** 结构。
5. 外层括号后再接 **[]**, 形成 **array** 后缀。

1.4.5 题 5 [SDD] C 声明英文说明

属性:

D.name: 综合属性, 标识符名。
D.desc: 综合属性, 英文说明片段。

规则:

```
D -> id      D.name = id.lexeme; D.desc = ""
D -> D1 ( )   D.name = D1.name; D.desc = D1.desc + " function returning"
D -> D1 []    D.name = D1.name; D.desc = D1.desc + " array"
D -> * D1     D.name = D1.name; D.desc = D1.desc + " pointer to"
D -> ( D1 )   D.name = D1.name; D.desc = D1.desc
```

输出:

```
declare D.name as D.desc
```

1.4.6 题 6 [后端] 目标代码生成填空

答题原则:

1. 先看右侧注释和前后寄存器使用。
2. 条件跳转对应 **if/else** 或 **while** 条件。
3. 函数调用前准备参数, 调用后取返回值。
4. **return** 前把返回值放到约定位置。
5. 分支标签必须与控制流一致。

1.5 2022

1.5.1 题 1 [自动机] $((\epsilon \mid a)b^*)^*$

答案写法:

1. $\epsilon \mid a$ 用选择模板。
2. b^* 用闭包模板。
3. 连接为 $(\epsilon \mid a)b^*$ 。
4. 最外层再加 * 。
5. 子集构造按 `epsilon-closure` 和 `move` 列表。
6. DFA 最小化从终态/非终态分组开始。

1.5.2 题 2 [LL] 声明列表文法

题干:

```
D -> T L
T -> int | real
L -> L , id | id
```

消除左递归:

```
D -> T L
T -> int | real
L -> id L'
L' -> , id L' | epsilon
```

FIRST:

```
FIRST(D) = {int, real}
FIRST(T) = {int, real}
FIRST(L) = {id}
FIRST(L') = {comma, epsilon}
```

FOLLOW:

```
FOLLOW(D) = {$}
FOLLOW(T) = {id}
FOLLOW(L) = {$}
FOLLOW(L') = {$}
```

预测表:

```

M[D,int]  = D -> T L
M[D,real] = D -> T L
M[T,int]  = T -> int
M[T,real] = T -> real
M[L,id]   = L -> id L'
M[L',comma] = L' -> , id L'
M[L', $]   = L' -> epsilon

```

结论:

表中无多重入口, G' 是 $LL(1)$ 。

1.5.3 题 3 [ANTLR] $1 + 2 * 3 + 4 * 5$

优先级:

```

* 高于 +
+ 左结合

```

语法树结构:

$(1 + (2 * 3)) + (4 * 5)$

解释:

先归约两个乘法子树, 再按 $+$ 的左结合形成外层加法树。

1.5.4 题 4 [LR] LR(1) 自动机与分析表

答题模板:

1. 写增广文法 $S' \rightarrow S$ 。
2. 求 FIRST 和 FOLLOW。
3. 从 $[S' \rightarrow \cdot S, \$]$ 做 closure。
4. 按题目要求的符号顺序计算 goto。
5. 归约项 $[A \rightarrow \alpha \cdot, a]$ 只在 a 列放 reduce。
6. 表中无冲突就是 LR(1)。

1.5.5 题 5 [SDD] 声明个数与嵌套深度

题干:

```

P -> D
D -> D ; D
D -> T id
D -> func id {D}
T -> int

```

统计声明个数:

P -> D	P.count = D.count
D -> D1 ; D2	D.count = D1.count + D2.count
D -> T id	D.count = 1
D -> func id {D1}	D.count = 1 + D1.count
T -> int	无需规则

打印变量嵌套深度:

P -> D	D.depth = 0
D -> D1 ; D2	D1.depth = D.depth; D2.depth = D.depth
D -> T id	print(id, D.depth)
D -> func id {D1}	D1.depth = D.depth + 1
T -> int	无需规则

1.5.6 题 6 [TAC] GCD while 回填

题干:

```

while a != b do
  if a > b then
    a = a - b
  else
    b = b - a

```

中间代码:

```

100: if a != b goto 102
101: goto 108
102: if a > b goto 104
103: goto 106
104: a = a - b
105: goto 100
106: b = b - a
107: goto 100
108: ...

```

回填说明:

while 条件真链回填到 102。
 while 条件假链回填到 108。
 if 条件真链回填到 104。
 if 条件假链回填到 106。
 两个分支结束后跳回 100。

1.6 2021

1.6.1 题 1 [自动机] ϵ | (a* b)

答案写法:

1. ϵ 与 a* b 做选择。
2. a* 先按闭包构造。
3. a* 与 b 连接。
4. 整体与 ϵ 选择。
5. 再做子集构造和最小化。

1.6.2 题 2 [LL] $S \rightarrow AB, A \rightarrow Aaa \mid a, B \rightarrow Bbb \mid b$

消除左递归:

$S \rightarrow AB$
 $A \rightarrow aA'$
 $A' \rightarrow aA' \mid \epsilon$
 $B \rightarrow bB'$
 $B' \rightarrow bB' \mid \epsilon$

FIRST:

$FIRST(S) = \{a\}$
 $FIRST(A) = \{a\}$
 $FIRST(A') = \{a, \epsilon\}$
 $FIRST(B) = \{b\}$
 $FIRST(B') = \{b, \epsilon\}$

FOLLOW:

$FOLLOW(S) = \{\$ \}$
 $FOLLOW(A) = \{b\}$
 $FOLLOW(A') = \{b\}$
 $FOLLOW(B) = \{\$ \}$
 $FOLLOW(B') = \{\$ \}$

预测表核心项:

```

M[S,a] = S -> A B
M[A,a] = A -> a A'
M[A',a] = A' -> a a A'
M[A',b] = A' -> epsilon
M[B,b] = B -> b B'
M[B',b] = B' -> b b B'
M[B',\$] = B' -> epsilon

```

结论:

表中无多重入口, G' 是 $LL(1)$ 。

1.6.3 题 3 [LR] $S \rightarrow V=E \mid E, E \rightarrow V, V \rightarrow x \mid *E$

答案模板:

1. 写 FIRST/FOLLOW。
2. 构造 $LR(1)$ 项目集。
3. 对 $[A \rightarrow \alpha \cdot, a]$ 按 a 设置归约。
4. 给出 $LR(1)$ 表并检查冲突。
5. 对输入 $*x=**x$ 按状态栈、符号栈、输入串写移入/归约过程。
6. LALR(1) 合并同心项目集后检查冲突。

1.6.4 题 4 [SDD] 逗号与嵌套输出

答案模板:

定义综合属性 $L.list$, 保存从左到右收集到的 a 的编号。
 定义继承属性 $L.base$, 表示当前嵌套或当前位置基准。
 每遇到一个 a , 就把当前编号加入输出。
 遇到嵌套结构时, 把基准传入内部子树。

1.6.5 题 5 [TAC] do-while 回填

模板:

```

do S while B

M1 = 循环体入口
翻译 S
M2 = 条件入口
backpatch(S.nextlist, M2)
翻译 B
backpatch(B.truelist, M1)
S.nextlist = B.falselist

```

1.6.6 题 6 [CFG] Chomsky Normal Form

CNF 要求:

$S \rightarrow \epsilon$ 只允许开始符号产生空串。

$A \rightarrow BC$, 其中 B 、 C 是非终结符。

$A \rightarrow a$, 其中 a 是终结符。

改写步骤:

1. 新增开始符号 $S_0 \rightarrow S$ 。
2. 删除 ϵ 产生式。
3. 删除单产生式 $A \rightarrow B$ 。
4. 删除无用符号。
5. 长右部拆成二元产生式。
6. 混合右部中的终结符替换成新非终结符。

1.7 2020

1.7.1 [自动机][LL][LR][SDD][后端] 稳定题型

2020 卷面覆盖的稳定考法按以下模板作答:

自动机: Thompson、子集构造、DFA 最小化。

LL: FIRST、FOLLOW、预测表、LL(1) 判断。

LR: 项目集、closure、goto、分析表、冲突判断。

SDD: 定义属性, 逐产生式写语义规则。

后端: 按控制流、调用约定、寄存器和返回值填目标代码。

后端填空顺序:

1. 标出基本块和分支标签。
2. 看每条缺失指令前后的寄存器活跃值。
3. 条件语句填 `compare` 和 `branch`。
4. 函数调用填参数传递、`call`、返回值接收。
5. `return` 前恢复栈帧并返回。